

Prosjektplan

VideoAPI for HDO

Mellomvare for standardisert videokommunikasjon i helsetjenesten

Nahom Berhane - 106371

Fredrik Andreas- 116143

Tim Harseth - 116177

Bachelor i programmering
Institutt for informasjonsteknologi
NTNU Gjøvik

Versjon 1.0
26. februar 2026

Veileder:	Sony George
Oppdragsgiver:	Helsetjenestens driftsorganisasjon (HDO)
Kontaktperson:	Magne Henriksen
Prosjektperiode:	12. januar - 20. mai 2026

Innhold

1	MÅL OG RAMMER	4
1.1	Bakgrunn	4
1.2	Eksisterende løsninger	4
1.3	Prosjektmål	4
1.3.1	Effektmål	4
1.3.2	Resultatmål	5
1.3.3	Læringsmål	5
1.4	Rammer	6
1.4.1	Tidsramme	6
1.4.2	Ressurser	6
1.4.3	Budsjett	6
1.4.4	Juridiske rammer	6
1.4.5	Teknologiske rammer	6
2	OMFANG	8
2.1	Problemområde/Fagfelt	8
2.2	Avgrensning	8
2.2.1	Hva som er med i prosjektet	8
2.2.2	Hva som ikke er med i prosjektet	9
2.3	Oppgavebeskrivelse	9
2.3.1	Hovedoppgave	9
2.3.2	Deloppgaver	10
2.3.3	Suksesskriterier	10
3	PROSJEKTORGANISERING	11
3.1	Ansvarsforhold og roller	11
3.1.1	Studentteam	11
3.1.2	Felles ansvar	11
3.1.3	Ekstern organisasjon	12
3.1.4	Organisasjonskart	12
3.2	Rutiner og gruppereregler	12
3.2.1	Arbeidsrutiner	12
3.2.2	Møteplan	13
3.2.3	Kommunikasjonsrutiner	13
3.2.4	Konflikthåndtering	14
3.2.5	Forventninger til hverandre	14
3.2.6	Konkrete gruppereregler	14
4	PLANLEGGING, OPPFØLGING OG RAPPORTERING	15
4.1	Hovedinndeling av prosjektet	15
4.2	Valg av prosessrammeverk	15
4.3	Milepæler	16
4.4	Plan for statusmøter og beslutningspunkter	16
4.5	Akseptansekriterier og kvalitetssikring	16

5	ORGANISERING AV KVALITETSSIKRING	17
5.1	Dokumentasjon, standarder og verktøy	17
5.1.1	Dokumentasjon	17
5.1.2	Kodestandarder	17
5.1.3	Bruk av AI-verktøy	17
5.1.4	Versjonskontroll (Git)	17
5.1.5	GitHub Actions	17
5.1.6	Konfigurasjonsstyring	18
5.1.7	Verktøy	18
5.2	Plan for inspeksjoner og testing	19
5.2.1	Code review / inspeksjoner	19
5.2.2	Teststrategi	19
5.3	Risikoanalyse på prosjektnivå	20
5.3.1	Tiltak for Risiko	20
6	PLAN FOR GJENNOMFØRING	21
6.1	Gantt-skjema	21
6.2	Aktivitetsliste	21
6.3	Milepæler	21
6.4	Kritisk sti	22
6.5	Buffer	22
6.6	Avhengigheter	22
6.7	Gantt-diagram	23
6.8	Kravspesifikasjon	25
	Referanser	32

1 MÅL OG RAMMER

1.1 Bakgrunn

Helsetjenestens driftsorganisasjon for nødnett HF (HDO) leverer kritiske kommunikasjonsløsninger til helsetjenesten i Norge. Organisasjonen drifter i dag videoløsninger for 109 kontrollrom [1] spredt over hele landet, hvor akuttmedisinske kommunikasjonsentraler (AMK) og legevaktsentraler (LVS) benytter video som støtteverktøy i akutte situasjoner.

Dagens løsning baserer seg på AntMedia Server som videomotor [1]. HDO ønsker nå å kunne benytte NHN Video (basert på Pexip) som alternativ, og samtidig legge til rette for at nye videomotorer enkelt kan integreres i fremtiden. Utfordringen er at hver videomotor har sitt eget API og sine egne integrasjonsmetoder, noe som fører til:

- Dobbelte vedlikehold og opplæring av driftspersonell
- Ulik brukeropplevelse mellom løsningene
- Kompleksitet ved utvikling av ny funksjonalitet
- Høy terskel for å bytte leverandør eller legge til nye videomotorer

Videoløsningen benyttes daglig i akuttmedisinsk kontekst hvor operatør sender SMS til innringer/pasient for å etablere videostrømming. Kun video overføres via løsningen, mens lyd går via ordinær telefonsamtale. Video anvendes som beslutningsstøtte i akuttmedisinske hendelser ved blant annet hjertestans, trafikkulykker og fødsler. Løsningen utgjør dermed kritisk infrastruktur hvor kravene til stabilitet, sikkerhet og tilgjengelighet er særlig høye.

For å møte disse utfordringene ønsker HDO en standardisert mellomware (VideoAPI) som kan fungere som felles integrasjonspunkt mot ulike videomotorer. Dette bachelorprosjektet skal utvikle en slik mellomware som proof-of-concept, med integrasjon mot AntMedia Server og NHN Video.

1.2 Eksisterende løsninger

Litteratursøk viser at det ikke finnes eksisterende løsninger som kombinerer AntMedia Server og NHN Video i en unified middleware. Prosjektet representerer derfor en ny løsning på HDOs integrasjonsbehov.

1.3 Prosjektmål

1.3.1 Effektmål

VideoAPI-løsningen skal gi HDO følgende langsiktige gevinster:

- **Redusert implementeringstid:** Nye videomotorer skal kunne integreres betydelig raskere og med lavere kompleksitet gjennom plugin-arkitekturen
- **Leverandøruavhengighet:** Mulighet til å bytte eller legge til leverandører basert på pris, kvalitet eller funksjonalitet

- **Fremtidig utvidbarhet:** Arkitektur som legger til rette for nye integrasjoner og tjenester
- **Forenklet vedlikehold:** Redusert kompleksitet i HDOs systemlandskap

Viktig avgrensning: Dette prosjektet leverer en proof-of-concept. Realisering av effektmålene krever videreutvikling og produksjonssetting.

1.3.2 Resultatmål

Prosjektet leverer følgende konkrete resultater:

1. VideoAPI mellomvare (REST API med plugin-arkitektur, OAuth2/JWT, health-endepunkter, logging)
2. AntMedia og NHN Video integrasjoner
3. Demo-frontend
4. API-dokumentasjon (OpenAPI/Swagger)
5. Teknisk dokumentasjon (README, deployment-guide, utviklerguide)
6. Docker image og CI/CD pipeline
7. Bachelorrapport

Frister: Produktleveranse 1. mai, rapportinnlevering 20. mai 2026.

1.3.3 Læringsmål

Gjennom prosjektet skal teamet oppnå kompetanse innen:

Teknisk kompetanse:

- C# og .NET 10 (nytt for teamet)
- ASP.NET Core Web API-utvikling
- REST API-design og implementasjon
- Plugin-arkitektur og modulære systemer
- WebRTC og videostrømming-teknologi
- Docker, containerisering og CI/CD (GitHub Actions)

Prosess og samarbeid:

- Samarbeid med ekstern oppdragsgiver
- Scrum i praksis med sprinter og leveranser

Teamet har ikke tidligere erfaring med C# eller .NET-økosystemet, noe som gjør dette til et særlig viktig læringsområde i prosjektet. Tilegnelse av denne kompetansen er planlagt inn som en del av Sprint 1-2.

1.4 Rammer

1.4.1 Tidsramme

Prosjektet gjennomføres i perioden 12. januar til 20. mai 2026 (18 uker):

- Produktleveranse: soft deadline 20. april, hard deadline 1. mai
- Rapportinnlevering: soft deadline 16. mai, hard deadline 20. mai

1.4.2 Ressurser

Team: Nahom Berhane, Fredrik Wiik, Tim Harseth (NTNU Gjøvik)

Veileder: Sony George (NTNU)

Oppdragsgiver: Magne Henriksen (HDO)

Møter: Veileder og oppdragsgiver annenhver uke

Testmiljøer fra HDO [1]:

- AntMedia: <https://bifrost-dev.hdo.no>
- NHN Video: <https://join.test.video.nhn.no>

1.4.3 Budsjett

Studentprosjekt uten budsjett. Alle nødvendige ressurser er tilgjengelig uten kostnad.

1.4.4 Juridiske rammer

Standardavtale NTNU-HDO signert 18. januar 2026 [2] (se Vedlegg A):

- Studenten eier resultatene
- HDO får ikke-eksklusiv bruksrett
- Åpen publisering etter 20. mai 2026

Signert NDA påvirker ikke offentliggjøring av oppgaven.

1.4.5 Teknologiske rammer

Anbefalt rammeverk: .NET 10 (ikke obligatorisk)

Obligatoriske tekniske krav [1]:

- Containerisering (Docker)
- Stateless arkitektur
- Konfigurasjon via miljøvariabler
- Token-basert autentisering (OAuth 2.0/JWT)
- API-dokumentasjon (OpenAPI/Swagger)

Valgt teknologistack:

- Backend: ASP.NET Core Web API (.NET 10)
- Autentisering: JWT Bearer Authentication
- Containerisering: Docker + Docker Compose
- CI/CD: GitHub Actions
- Testing: xUnit
- API-dokumentasjon: Swashbuckle (Swagger/OpenAPI)

2 OMFANG

2.1 Problemområde/Fagfelt

Prosjektet omhandler utvikling av teknisk infrastruktur for sanntids videoløsninger i helse-tjenesten. Bakgrunnen er at HDO i dag benytter to separate videoløsninger som i stor grad dekker samme funksjonelle behov, men som er basert på ulike teknologisk infrastruktur. Dette skaper utfordringer knyttet til integrasjon, videreutvikling og vedlikehold, og gir behov for en felles, standardisert tilnærming.

Arbeidet plasserer seg innenfor følgende fagområder:

- videoløsninger i helsetjenesten, der video brukes som beslutningsstøtte i akuttmedisinsk kontekst
- API-design og mellomvarearkitektur, med fokus på en standardisert kobling mellom systemer
- WebRTC-teknologi for sanntids videostrømming
- backend-integrasjon mot eksterne videoplattformer
- sikkerhet i kritisk infrastruktur med autentisering, tilgangskontroll og logging

Prosjektet kombinerer dermed systemutvikling, distribuert arkitektur og sikkerhetsmekanismer i en helsefaglig kontekst.

2.2 Avgrensning

Prosjektets omfang er avgrenset for å sikre gjennomførbarhet innenfor rammene av et bachelorprosjekt, og tar utgangspunkt i kravene definert av HDO [1].

2.2.1 Hva som er med i prosjektet

Prosjektet omfatter utvikling av en mellomvare (VideoAPI) som eksponerer et standardisert REST-basert API mot underliggende videomotorer. Arbeidet inkluderer:

Videomotor-integrasjoner:

- AntMedia Server
- NHN Video (Pexip)

Sikkerhet:

- OAuth 2.0 og JWT-autentisering for API
- Sikkerhetstoken per videorom

Funksjonalitet:

- Opprettelse og sletting av videorom
- Start og stopp av videostrøm
- Henting av roominformasjon

Observability:

- Health-endepunkter (/health/live, /health/ready)
- Logging av drifts- og sikkerhetsrelevante hendelser

Demonstrasjon og dokumentasjon:

- Enkel demo-frontend for funksjonell verifisering
- API-dokumentasjon (OpenAPI/Swagger)
- Teknisk dokumentasjon (README, deployment-guide, utviklerguide)

Deployment:

- Docker containerisering
- CI/CD pipeline (GitHub Actions)

Disse elementene dekker både funksjonelle og ikke-funksjonelle krav definert i kravspesifikasjonen [1].

2.2.2 Hva som ikke er med i prosjektet

Prosjektet omfatter ikke implementering eller bruk av VideoAPI i eksisterende HDO-løsninger, og inkluderer heller ikke produksjonssetting eller operativ drift. Videre er følgende eksplisitt avgrenset bort:

- Lagring av video, lyd, bilder eller pasientopplysninger
- Produksjonsgrad frontend (demo-frontend inngår)
- Håndtering av lyd (kun video overføres)
- SMS-funksjonalitet (utenfor scope)
- Brukeradministrasjon (autentisering inngår, men ikke bruker-CRUD)
- Støtte for andre videomotorer enn AntMedia og NHN Video
- Opptak eller avspilling av video

Disse avgrensningene er nødvendige for å begrense kompleksitet og sikre at prosjektet kan gjennomføres innen tilgjengelig tid og ressurser.

2.3 Oppgavebeskrivelse

2.3.1 Hovedoppgave

Hovedoppgaven er å utvikle et sikkert og modulært VideoAPI som fungerer som et felles integrasjonspunkt mellom HDO sine videoløsninger og ulike underliggende videomotorer. Løsningen skal standardisere samhandlingen med videoplattformene og legge til rette for fremtidige utvidelser uten omfattende endringer i eksisterende kode.

2.3.2 Deloppgaver

Prosjektet er delt inn i følgende hovedoppgaver (se Gantt-diagram 6.7. for tidsplan):

1. Kravanalyse basert på oppgavebeskrivelse og kravspesifikasjon
2. Arkitektur- og designarbeid
3. Implementasjon av kjernefunksjonalitet i VideoAPI
4. Implementasjon av backend-integrasjoner (AntMedia, NHN)
5. Implementasjon av sikkerhet og observability
6. Utvikling av demo-frontend og tilhørende dokumentasjon
7. Testing (kontinuerlig gjennom utviklingen)
8. Containerisering og deployment-klargjøring
9. Utarbeidelse av bachelorrapport

2.3.3 Suksesskriterier

Prosjektet anses som vellykket dersom:

- **Funksjonelle og ikke-funksjonelle krav:** Alle krav definert i kravspesifikasjonen [1] (F1.1-F1.6, S1.1-S1.2, I1.1-I1.2, O1.1-O1.2, D1.1) er implementert og verifisert
- **Demonstrasjon:** Demo-frontend kan demonstrere følgende funksjonalitet:
 - Opprettelse av videorom
 - Oppkobling til rom (URL/token)
 - Start og stopp av videostrøm
 - Visning av roominformasjon
- **Dokumentasjon:** Komplette API-dokumentasjon (OpenAPI/Swagger) og teknisk dokumentasjon (README, deployment-guide, utviklerguide)
- **Godkjenning fra oppdragsgiver:** HDO vurderer leveransen som tilfredsstillende i siste sprint review (innen 1. mai 2026)
- **Akademisk godkjenning:** Bachelorrapporten godkjennes av NTNU i henhold til gjeldende krav

3 PROSJEKTORGANISERING

3.1 Ansvarsforhold og roller

3.1.1 Studentteam

Studentteamet består av tre studenter. Ansvar og roller er fordelt basert på kompetanse og interesseområder. Rollene angir primæransvar, men alle gruppemedlemmer bidrar ved behov på tvers av oppgaver.

Nahom Berhane - Scrum Master og Kommunikasjonsansvarlig

- Fasilitering av scrum-seremonier (stand-up, sprint planning, retrospektiv)
- Kontaktperson mot HDO og veileder, oppretter møteagendaer
- Backend-utvikling, API-design og systemarkitektur

Fredrik Wiik - Utviklingsansvarlig

- Backend- og frontend-utvikling
- Infrastruktur og deployment
- Docker containerisering

Tim Harseth - Kvalitetssikringsansvarlig og DevOps

- Backend-utvikling
- Testing (enhetstester, integrasjonstester)
- CI/CD pipeline (GitHub Actions)
- Ukentlig kvalitetssjekk av rapport og kode

Rollene er ment å sikre tydelig ansvarsfordeling samtidig som gruppen legger til rette for samarbeid og kunnskapsdeling.

3.1.2 Felles ansvar

I tillegg til individuelle roller har alle gruppemedlemmene et felles ansvar for sentrale oppgaver i prosjektet:

- **Rapportskriving:** Alle gruppemedlemmer bidrar til planlegging, utarbeidelse og kvalitetssikring av bachelor rapporten.
- **Code review:** All kode som merges til hovedgren gjennomgår code review av minst ett annet gruppemedlem for å sikre kvalitet, lesbarhet og korrekt funksjonalitet.
- **Testing:** Gruppen deler ansvar for planlegging og gjennomføring av testing, inkludert enhetstester, integrasjonstester og manuell verifikasjon av funksjonalitet.
- **Møtereferater:** Alle gruppemedlemmer deler ansvar for dokumentasjon av møter. Ansvar roterer mellom medlemmene, og referater dokumenteres i Notion.

- **Møteforberedelser:** Alle deltar aktivt i forberedelse til møter, herunder gjennomgang av status, identifisering av utfordringer og forberedelse av spørsmål til veileder og oppdragsgiver.
- **Kommunikasjon med oppdragsgiver:** Gruppen har et felles ansvar for profesjonell og tydelig kommunikasjon med oppdragsgiver, der relevant informasjon deles internt i gruppen.

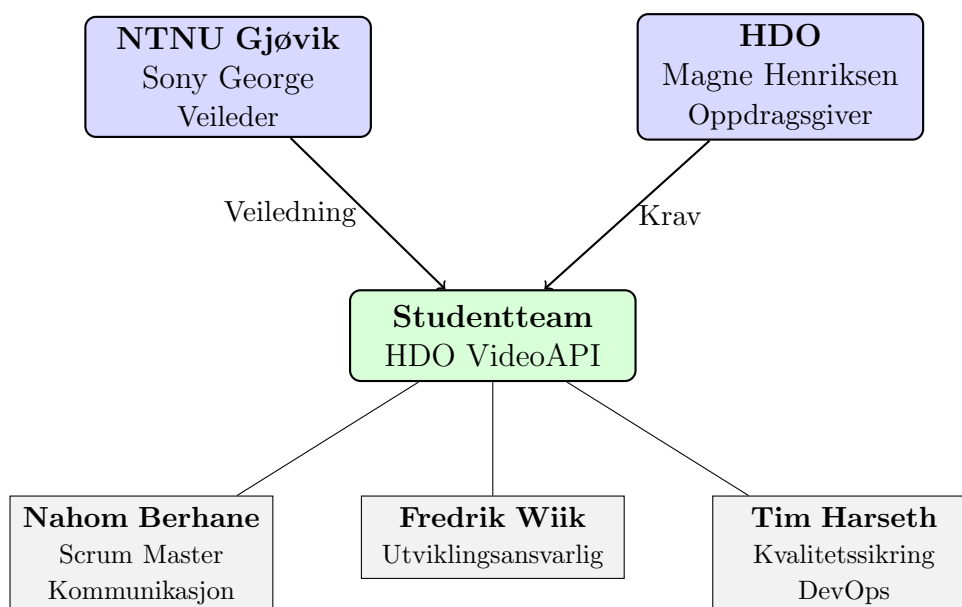
3.1.3 Ekstern organisasjon

Veileder Veileder for prosjektet er Sony George. Veileder har ansvar for faglig og metodisk veiledning, samt å gi tilbakemeldinger på prosjektets gjennomføring og rapportarbeid. Det planlegges jevnlig møter med veileder, hver uke, samt ved behov.

Oppdragsgiver Oppdragsgiver for prosjektet er HDO, representert ved Magne Henriksen. Oppdragsgiver bidrar med domenekunnskap, kravavklaringer og faglige innspill knyttet til problemområdet. Det avholdes møter med oppdragsgiver etter behov, hovedsakelig i forbindelse med avklaringer, milepæler og gjennomgang av løsningen.

Studentgruppen har det overordnede ansvaret for prosjektets faglige innhold, valg av løsninger og gjennomføring.

3.1.4 Organisasjonskart



Figur 1: Organisasjonskart for HDO VideoAPI-prosjektet

3.2 Rutiner og grupperegler

3.2.1 Arbeidsrutiner

Prosjektet gjennomføres etter en lett smidig metodikk basert på Scrum, tilpasset prosjektets omfang og gruppens størrelse. Det benyttes sprinter med varighet på to uker. Hver sprint inkluderer planlegging, gjennomføring og evaluering av arbeidet.

Gruppen møtes mandag og tirsdag mellom kl. 09:00-12:00 for statusoppdatering, koordinering og felles arbeid etter behov. I tillegg gjennomføres møter ved behov i løpet av

uken, blant annet i forbindelse med avklaringer og problemløsning. Sprint review og retrospektiv gjennomføres ved slutten av hver sprint. Pair programming benyttes ved behov, og all kode som merges til hovedgren gjennomgår code review.

3.2.2 Møteplan

Interne møter:

- **Gruppemøter:** Mandag og tirsdag kl. 09:00-12:00 - status, koordinering og felles arbeid etter behov
- **Sprint planning:** Hver 2. uke r) - planlegging av neste sprint
- **Sprint retrospective:** Hver 2. uke (oddetallsuker) - evaluering og forbedring

Eksterne møter:

- **Veiledermøter:** Ukentlig (dag og tid avtales)
- **Møter med HDO (Sprint review):** Annenhver uke - demonstrasjon og tilbakemeldinger

Arbeidstid:

- Minimum 30 timer/uke per person
- Kjernetid for møter: Mandag og tirsdag kl. 09:00-12:00

3.2.3 Kommunikasjonsrutiner

Gruppen benytter følgende verktøy for kommunikasjon og samarbeid:

Verktøy	Bruksområde
Discord	Daglig kommunikasjon, raske spørsmål, uformelle møter
Microsoft Teams	Formelle møter med veileder og HDO
Notion	Møtereferater, intern dokumentasjon, kunnskapsdeling
Jira	Sprint planning, oppgavestyring, backlog
Overleaf	Samskriving av prosjektplan og bachelorrapport
GitHub	Kode, versjonskontroll, code review

Tabell 1: Kommunikasjonsverktøy i prosjektet

Dokumentasjonsrutiner:

- Møtereferater føres i Notion
- Viktige beslutninger dokumenteres skriftlig samme dag
- Ukentlig oppdatering av timelog i Notion (fredag kl. 16:00)

3.2.4 Konfliktbehandling

Eventuelle konflikter forsøkes først løst internt i gruppen gjennom dialog. Dersom dette ikke fører frem, involveres veileder for bistand. Ved alvorlige eller vedvarende konflikter kan saken eskaleres i tråd med NTNUs retningslinjer.

3.2.5 Forventninger til hverandre

Gruppemedlemmene forventes å møte til avtalt tid, gjennomføre tildelte oppgaver innen frist og være åpne om utfordringer som kan påvirke fremdriften. Det forventes en respektfull og profesjonell samarbeidsform. Meldinger besvares normalt samme dag.

3.2.6 Konkrete grupperegler

Fremmøte og punktlighet:

- Alle møter til avtalt tid
- Ved forsinkelse: Varsle gruppen senest 30 minutter før møtestart
- Ved fravær: Varsle minst 24 timer i forveien (akutt sykdom unntas)

Arbeidsforpliktelser:

- Minimum 30 timer arbeid per uke per person
- Tildelte oppgaver gjennomføres innen avtalt frist
- Ved forsinkelse: Varsle gruppen minst 2 dager før deadline

Kommunikasjon:

- Meldinger (Discord/Teams) besvares samme dag
- Viktige beslutninger dokumenteres skriftlig (Notion)

Sanksjoner ved brudd:

- Første gang: Intern samtale i gruppen
- Andre gang: Involvering av veileder
- Ved vedvarende brudd: Formelle tiltak i henhold til NTNUs regelverk

4 PLANLEGGING, OPPFØLGING OG RAPPORTERING

4.1 Hovedinndeling av prosjektet

Prosjektet deles inn i fem hovedfaser: analyse og krav, arkitektur og design, implementasjon, testing samt dokumentasjon og rapportering. Denne inndelingen skal sikre strukturert fremdrift samtidig som prosjektet kan gjennomføres iterativt.

Analysefasen tar utgangspunkt i kravspesifikasjonen mottatt fra HDO [1], supplert med gjennomgang av API-dokumentasjon for AntMedia og NHN Video, samt egen testing og utforskning. Målet er å etablere et tydelig og realistisk grunnlag for videre utvikling.

Basert på analysen utarbeides systemets arkitektur og design. Sentrale arkitekturvalg dokumenteres, og løsningen beskrives ved hjelp av overordnede arkitekturfigurer og sekvensdiagrammer som illustrerer flyten av API-kall.

Implementasjonen gjennomføres iterativt og deles inn i faser og sprints. Arbeidet omfatter utvikling av et grunnleggende API, støtte for flere videomotorer, sikkerhetsmekanismer, logging og helsesjekk, samt en enkel frontend for demonstrasjonsformål.

Testing utføres kontinuerlig gjennom prosjektet og består av en kombinasjon av enhets-tester, enkle integrasjonstester og manuell testing mot tilgjengelige testmiljøer. Dokumentasjon og rapportering skjer parallelt med utviklingsarbeidet, og ferdigstilles hovedsakelig mot slutten av prosjektperioden.

Detaljert tidsplan: Se Gantt-diagram i seksjon 6.7.

4.2 Valg av prosessrammeverk

Prosjektet gjennomføres ved bruk av Scrum som prosessrammeverk. Valget begrunnes med at kravene ikke er fullt ut avklart fra start, og det foreligger usikkerhet knyttet til integrasjon mot eksterne API-er (AntMedia, NHN Video). Scrum legger samtidig til rette for hyppig testing og justering underveis.

Scrum benyttes i en tilpasset form med to-ukers sprints. Prosjektet består av 9 sprints over 18 uker:

- **Sprint 1-2 (Uke 1-4):** Oppstart, analyse og planlegging
- **Sprint 3-4 (Uke 5-8):** Arkitektur, design og kjerne-API
- **Sprint 5-6 (Uke 9-12):** Backend-integrasjoner (AntMedia, NHN)
- **Sprint 7 (Uke 13-14):** Demo og observability
- **Sprint 8-9 (Uke 15-18):** Stabilisering, testing og rapportferdigstillelse

Arbeidet organiseres rundt konkrete sprintmål og en backlog, håndtert gjennom Jira. Hver oppgave trackes som issue med status, prioritet og ansvarlig. Alle gruppe-medlemmene deltar som utviklere, mens oppdragsgiver fra HDO fungerer som produkteier og bidrar med kravavklaringer og prioriteringer ved behov.

Prosessrammeverket tilpasses prosjektets kontekst, med hovedfokus på læring, oversikt og fremdrift fremfor full etterlevelse av formelle Scrum-seremonier.

4.3 Milepæler

Prosjektet har følgende sentrale milepæler:

ID	Uke	Milepæl
M1	4	Prosjektplan godkjent
M2	7	Første fungerende API
M3	11	Begge videomotorer integrert
M4	13	Demo-frontend ferdig
M5	16	Produktleveranse (soft deadline)
M6	18	Rapport ferdig (soft deadline)
M7	19	Endelig innlevering

Tabell 2: Prosjektets milepæler

Milepælene markerer kritiske beslutningspunkter og leveranser. Ved hver milepæl gjennomføres evaluering av status og fremdrift.

4.4 Plan for statusmøter og beslutningspunkter

Prosjektets fremdrift følges opp gjennom interne møter mandag og tirsdag (kl. 09:00-12:00), samt møter med veileder ukentlig og oppdragsgiver annenhver uke. Møtene benyttes til å følge opp fremdrift, avklare utfordringer og planlegge videre arbeid.

Beslutningspunkter er knyttet til ferdigstillelse av analyse- og arkitekturfase, godkjenning av sentrale tekniske valg, samt evaluering av leveranser etter utvalgte sprinter. Dette bidrar til kontrollert fremdrift og sikrer at prosjektet holder ønsket faglig og teknisk retning.

Kritiske risikoer følges opp kontinuerlig, spesielt knyttet til den kritiske stien (se risikoanalyse i seksjon 3 og Gantt-diagram i seksjon 6.7).

4.5 Akseptansekriterier og kvalitetssikring

Hver leveranse evalueres mot definerte akseptansekriterier (se suksesskriterier i seksjon 2.3.3):

- **Funksjonelle krav:** Alle krav F1.1-F1.6 fra kravspesifikasjonen [1] må være implementert og verifisert
- **Sikkerhet:** OAuth2/JWT-autentisering (S1.1) og romtoken (S1.2) må være implementert
- **Infrastruktur:** Docker-containerisering (I1.1) og stateless design (I1.2) må være på plass
- **Observability:** Health-enderpunkter (O1.1) og logging (O1.2) må fungere
- **Dokumentasjon:** API-dokumentasjon (D1.1) må være komplett

Disse kriteriene gjennomgås ved hver sprint review med oppdragsgiver.

5 ORGANISERING AV KVALITETSSIKRING

5.1 Dokumentasjon, standarder og verktøy

5.1.1 Dokumentasjon

Rapport skrives i Latex med bruk av Overleaf, som gjør det lett for gruppemedlemmene å skrive samtidig. Overleaf dokumentet vil bli synkronisert med et separat GitHub Repo for ekstra backup.

Notion er et samarbeids notatverktøy som skal bli brukt for intern informasjon som møtereferater og generell informasjon som blir delt mellom gruppen.

5.1.2 Kodestandarder

Prosjektet følger i hovedsak Microsoft sine anbefalte konvensjoner for C#. Public typer og medlemmer navngis med PascalCase, mens private felt benytter camelCase. Det brukes file-scoped namespaces for en tydelig og moderne kodeorganisering.

Asynkrone metoder implementeres i tråd med etablerte async/await-prinsipper, og prosjektet unngår blokkering gjennom bruk av Task.Result og .Wait()

5.1.3 Bruk av AI-verktøy

I prosjektet benytter gruppen AI-verktøy som ChatGPT og Claude som støtte i arbeidsprosessen. ChatGPT brukes hovedsakelig til idéutvikling, strukturering av tekst og forbedring av dokumentasjon. Claude benyttes i tillegg som støtte ved idéutvikling og til hjelp med koding og tekniske problemstillinger. AI-verktøyene fungerer som assistenter i utviklingsarbeidet, og alle forslag blir vurdert og kvalitetssikret av gruppemedlemmene før de tas i bruk

5.1.4 Versjonskontroll (Git)

Prosjektet vil benytte Git og Github som versjonkontrollsystem. Dette vil sikre strukturert utvikling og samarbeid. Vi bruker en branch strategi hvor main vil være protected slik at man kun kan gjøre endringer i kode gjennom Pull requests der minst ett annet gruppe-medlem må godkjenne det. En develop branch som brukes til utvikling og integrasjon av koden, og spesifikke funksjoner utvikles i egne feature-branches.

Commit-meldinger skal være profesjonelle og forklare hver endring i kodebasen.

5.1.5 GitHub Actions

Prosjektet bruker Github, og for å sikre kontinuerlig kvalitet av kode, bruker vi GitHub Actions med automatiserte workflows som kjøres på en GitHub runner. Workflows vil aktivere ved push til repository og opprettelse av pull requests. Disse workflowene vil blant annet utføre:

- Kodevalidering
- Automatiserte tester
- Sjekk standarder og formatering

5.1.6 Konfigurasjonsstyring

Miljøspesifikk konfigurasjon håndteres som miljøvariabler, dette gjør at koden kan kjøres i ulike miljøer (lokalt, develop, main) uten kodeendringer. Miljøvariabler blir ikke lagret i repoet.

Hemmeligheter som tokens, API-nøkler og passord skal aldri lagres i koden eller i konfigurasjonsfiler i GitHub. I CI brukes GitHub Secrets for å legge inn nødvendige secrets ved kjøring av workflows. Tilgang til secrets vil endres og begrenses til de relevante workflows eller branches ved behov.

Avhengigheter vil håndteres gjennom pakkehåndtering NuGet og låses til versjoner for å sikre at koden kan kjøres overalt.

5.1.7 Verktøy

Navn	Beskrivelse	Bruksområde
Overleaf	Nettbasert LaTeX-editor for å skrive og kompilere dokumenter.	Prosjektrapport.
Draw.io	Gratis verktøy for å lage diagrammer og flytskjema.	UML og andre diagrammer.
Visual Studio Code	IDE for utvikling.	API-utvikling.
Docker	Plattform for å kjøre applikasjoner i containere.	Containerization av VideoAPI.
GitHub	Plattform for versjonskontroll og samarbeid med Git.	Lagring av kode, samarbeid og prosjektstyring.
Postman	Verktøy for testing av API-er.	Utvikling og feilsøking av REST API-er.
OpenStack	Infrastruktur som tjeneste (IaaS).	Hosting av GitHub Runner.
Jira	Prosjektstyringsverktøy med Scrumboard og issue tracking.	Prosjektstyring.
Teams	Kommunikasjons-applikasjon.	Samarbeid og kommunikasjon.
Notion	Nettbasert notatverktøy med samarbeidsmuligheter.	Dokumentasjon og notater.
ChatGpt 5.2	AI-språkmodell fra OpenAI	Ideutvikling og støtte i tekst
Claude	AI-språkmodell fra Anthropic	Ideutvikling og støtte i kode

5.2 Plan for inspeksjoner og testing

5.2.1 Code review / inspeksjoner

For å sikre høy kodekvalitet og redusere risiko for feil i produksjon, gjennomføres det code reviews som en fast del av utviklingsprosessen. Alle endringer i kodebasen skal gjøres gjennom Pull Requests i GitHub, og disse må godkjennes før de kan merges inn i main.

Code review utføres ved hver Pull Request, og minst ett annet gruppe-medlem må gjennomgå endringen før den godkjennes. Dette sikrer at koden blir kontrollert av flere og at kunnskapen om løsningen deles i teamet.

Ved inspeksjonene legges det særlig vekt på følgende:

- **Funksjonalitet:** At endringen løser oppgaven korrekt og ikke introduserer nye feil.
- **Lesbarhet og struktur:** At koden følger prosjektets kodestandarder og er lett å forstå og vedlikeholde.
- **Sikkerhet:** At endringer ikke skaper sikkerhets-hull, særlig knyttet til autentisering.
- **Ytelse:** At implementasjonen er effektiv og ikke gir unødvendig ressursbruk.

5.2.2 Teststrategi

Kode skal skrives med mål om å kunne lage og kjøre tester. Enhets-testing skal bli implementert på kritiske komponenter og funksjoner i kodebasen, og de ikke-kritiske komponentene hvis det er tid. I workflows vil disse enhets testene bli kjørt automatisk og må være godkjent før det er å lov å endre kode i develop og main branch.

Integrasjonstesting brukes for å verifisere at VideoAPI fungerer korrekt i samspill med de eksterne videomotorene, og det er derfor opprettet separate testmiljøer for begge motorene levert av oppdragsgiver. Testing i disse miljøene gjør det mulig å kontrollere at API-et håndterer kommunikasjon, responser og eventuelle feil fra videomotorene på en robust måte. Integrasjonstesting gjennomføres jevnlig gjennom utviklingsperioden, spesielt ved større endringer i funksjonalitet eller når nye endepunkter legges til. Dette sikrer at VideoAPI er kompatibel med begge videomotorene før løsning tas i bruk i produksjon.

5.3 Risikoanalyse på prosjektnivå

Tabell 3: Risikoanalyse – HDO VideoAPI

ID	Risiko	Sannsynlighet	Konsekvens	Tiltak
1	Et eller flere gruppemedlemmer kan bli syke eller utilgjengelige over lengre tid, noe som kan føre til redusert kapasitet og forsinkelser i utviklingsarbeidet.	Lav	Høy	Ja
2	Tap av kodebase eller rapport.	Lav	Høy	Ja
3	Gruppemedlemmer har konflikt.	Lav	Høy	Ja
4	Endringer i krav eller forventninger fra oppdragsgiver underveis kan føre til omprioritering og ekstra utviklingsarbeid.	Middels	Middels	Ja
5	Risiko for at prosjektet ikke ferdigstilles innen bachelorfristen, for eksempel på grunn av forsinkelser eller uforutsette tekniske utfordringer.	Lav	Høy	Ja
6	Videomotorene NHN og AntMedia går ned over lengre tid.	Lav	Høy	Begrenset

5.3.1 Tiltak for Risiko

ID 1 Siden pull requests må bli inspisert og godkjennes av et annet gruppemedlem, så slipper vi tap av fremgang hvis et medlem blir syk siden koden er kontrollert av flere.

ID 2 For å minimere risikoen for tap av kodebase og rapport, setter vi opp backups på begge og regelmessig oppdatere backups. Gjennom Overleaf sin Github Sync kan vi sette opp backup på et eget GitHub-repo.

ID 3 Eventuelle konflikter forsøkes først løst internt i gruppen gjennom dialog. Der- som dette ikke fører frem, involveres veileder for bistand. Ved alvorlige eller vedvarende konflikter kan saken eskaleres i tråd med NTNUs retningslinjer.

ID 4 For å redusere risikoen gjennomføres det jevnlig møter med oppdragsgiver for å avklare forventninger og oppdatere krav. Alle endringer dokumenteres fortløpende slik at teamet har en felles forståelse av prosjektets mål. Oppgaver prioriteres basert på prosjek- tets tidsramme og kjernefunksjonalitet. Dette sikrer at løsningen fortsatt leveres innen fristen selv ved justerte krav.

ID 5 Ha tydelig fremdriftsplan, prioritere kjernefunksjonalitet og følge opp deadlines gjennom sprint-planlegging.

6 PLAN FOR GJENNOMFØRING

6.1 Gantt-skjema

Prosjektets fremdrift er planlagt ved hjelp av et Gantt-diagram som viser hovedfaser, milepæler og tidsavhengigheter frem til innlevering. Diagrammet er lagt ved som vedlegg for bedre lesbarhet, se Vedlegg 6.7.

6.2 Aktivitetsliste

Tabell 4: Aktivitetsliste med varighet og ansvar.

ID	Aktivitet	Periode	Ansvar
A1	Oppstart og planlegging (møter, etablering av grunnlag)	Uke 1–3	Alle
A2	Analyse og krav (kravspec, API-dok, testing av testmiljø)	Uke 1–4	Alle
A3	Arkitektur og design (plugin/adapter, sekvens- og arkitekturdiagram)	Uke 4–5	Backend
A4	Kjerne-API og infrastruktur (REST, auth/JWT, health, Docker, CI)	Uke 6–7	Backend/DevOps
A5	AntMedia-integrasjon (adapter + endepunkter + verifikasjon)	Uke 8–9	Backend
A6	NHN-integrasjon (adapter + endepunkter + verifikasjon)	Uke 10–11	Backend
A7	Demo og observability (demo-frontend, logging, Swagger/API-dok)	Uke 12–13	Frontend/Backend
A8	Stabilisering og testing (bugfix, enhetstest, integrasjonstest, hardening)	Uke 14–16	DevOps/Alle
A9	Rapportskriving (fullføring av rapport)	Uke 16–18	Alle
A10	Buffer/innlevering (korrektur, formatering, siste sjekk)	Uke 18–19	Alle

6.3 Milepæler

Tabell 5: Milepæler i prosjektet.

ID	Milepæl	Tidspunkt
M1	Prosjektplan godkjent	Uke 4
M2	Første fungerende API (kjernefunksjonalitet tilgjengelig)	Uke 7
M3	Støtte for begge videomotorer (AntMedia + NHN)	Uke 11
M4	Demo-frontend ferdig (demonstrasjon av funksjonalitet)	Uke 13
M5	Produktleveranse (hard deadline)	Uke 16
M6	Rapport ferdig (soft deadline)	Uke 18
M7	Endelig innlevering (hard deadline)	Uke 19

6.4 Kritisk sti

Kritisk sti i prosjektet omfatter ferdigstillelse av arkitektur og design, etablering av kjerne-API med sikkerhet, samt integrasjon mot begge videomotorene. Forsinkelser i disse aktivitetene vil direkte påvirke produktleveransen 1. mai.

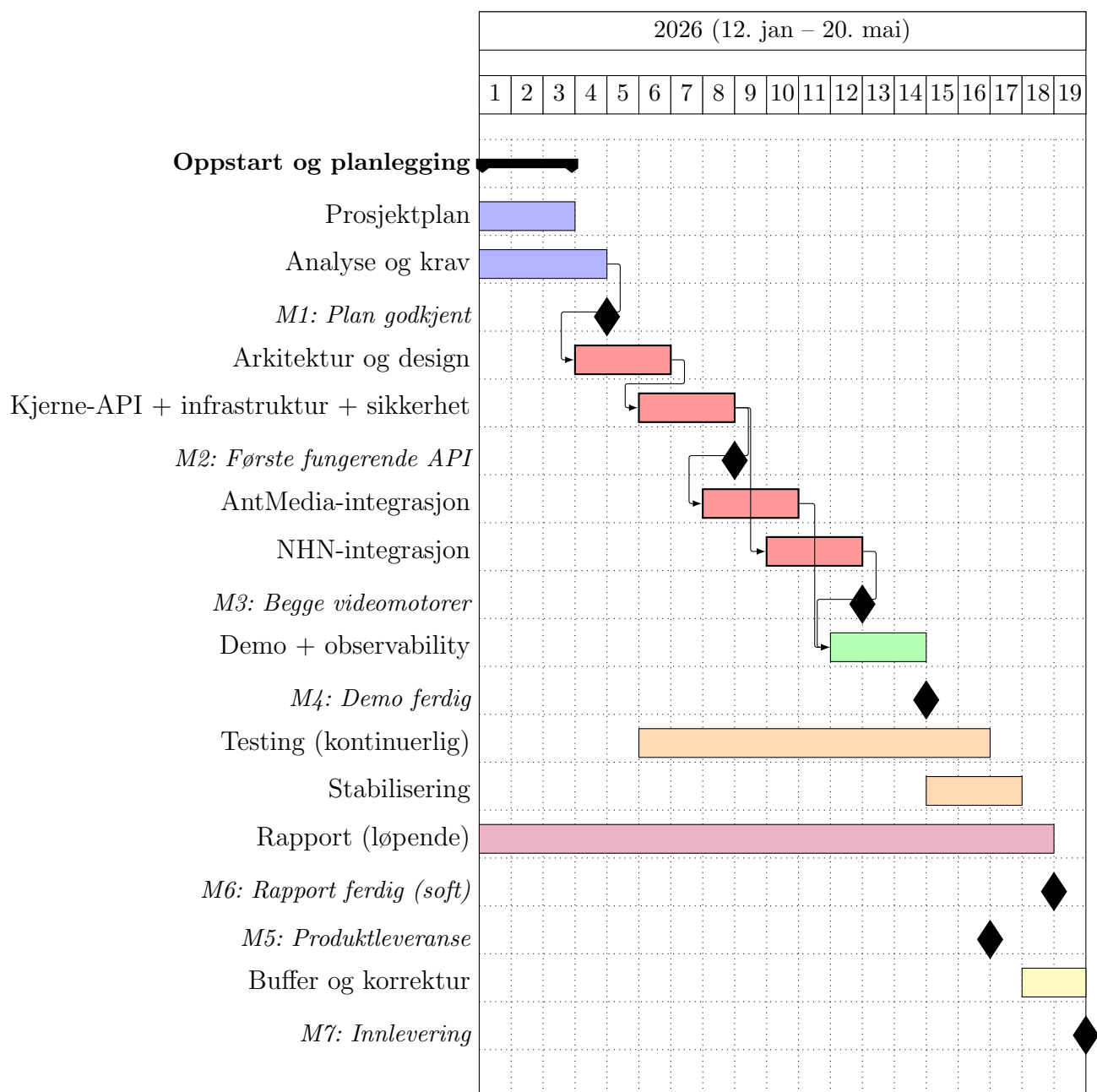
6.5 Buffer

Det er lagt inn buffer i perioden 16.–20. mai for korrektur, formatering og siste kvalitets sjekk før endelig innlevering.

6.6 Avhengigheter

Arkitektur og plugin-/adapterdesign må være ferdigstilt før backend-integrasjoner kan implementeres. Autentisering må være på plass før sikrede endepunkter ferdigstilles, og demo-frontend avhenger av stabil API-funksjonalitet.

6.7 Gantt-diagram



Figur 2: Gantt-diagram for HDO VideoAPI (uke 1–19).

Fargeforklaring:

- [Red] Kritisk sti
- [Purple] Analyse og design
- [Green] Implementasjon
- [Orange] Testing

-  Dokumentasjon

6.8 Kravspesifikasjon

HDO VideoAPI

Bakgrunn:

HDO har i dag 2 videoløsninger i drift. Begge løsningen operer på samme måte ved at operatør sender en SMS til innringer/pasient for å starte videostrømming.

Denne videostrømmen blir brukt som ett støtteverktøy for å ta rett beslutning i situasjonen.

For å få til bedre integrasjon og operabilitet mellom disse to løsningen er det ønskelig at de benytter samme rammeverk/videomotor

Videoløsningen brukes av 109 kontrollrom(AMK/LVS) over hele Norge.

Kravspesifikasjon:

Utvilke en mellomvare som tilbyr ett sett med standard APIer, som vil bli brukt av videoløsningene.

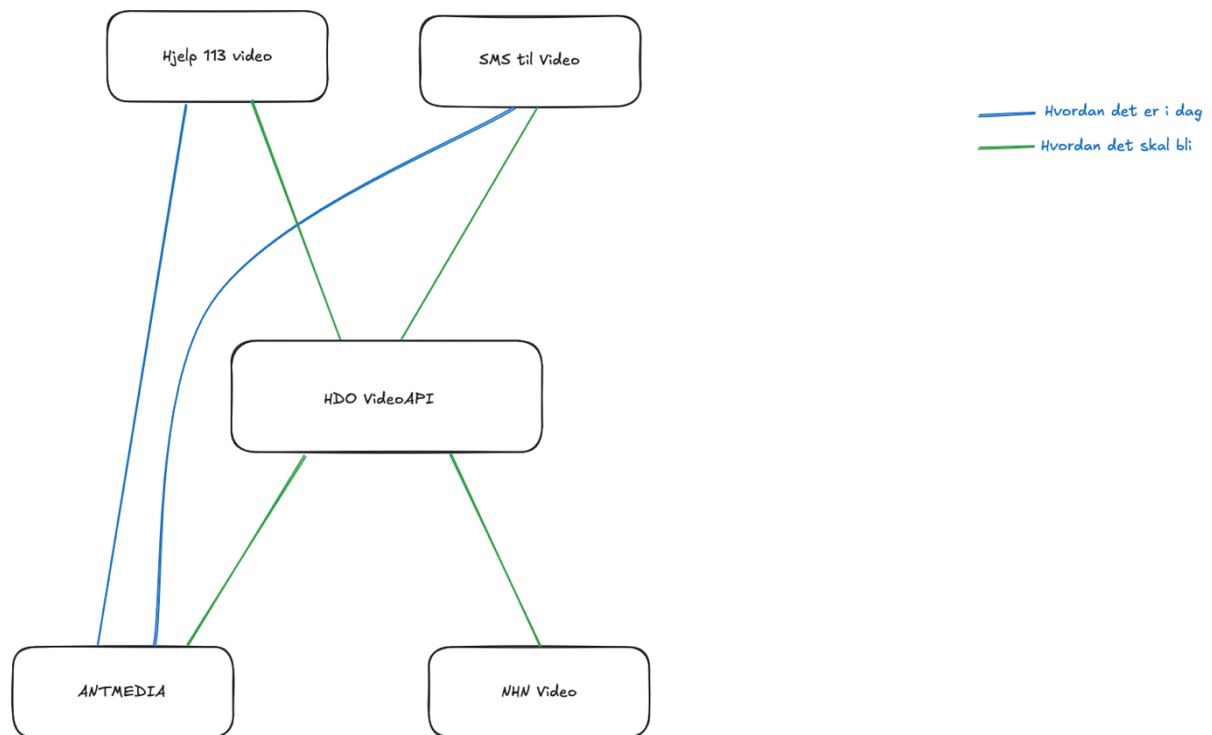
Mellomvaren håndterer innkommende API-kall og sender disse til korrekt/valgt videomotor/backend, for deretter returnere svar tilbake til løsningen.

Vi får nok ikke implementert denne i eksisterende videoløsninger iløpet av bachelorprosjekt perioden så det må antageligvis også lages en enkel "frontend" for å demonstrer funksjonalitet.

Mellomvaren må støtte følgende backend videomotorer:

- AntMedia
- NHN Video

Det bør også tenkes på eventuelle utvidelser og/eller endringer i videomotorer, slik at en ny backend kan implementeres uten å måtte skrive om hele mellomvaren



Figur 1 Konsept skisse over flyt

Krav:

S1.1	Sikkerhet	Autentisering	
S1.2	Sikkerhet	Opprettet token på videorom	Sette sikkerhets token på ett videorom, slik at ikke uvedkommende kan få tilgang til strømmen.
F1.1	Funksjonalitet	Oppretting av videorom.	Oppretter ett nytt videorom og returnere informasjon om dette. Om det ikke sendes med ett navn/id, må dette settes av

			mellomvaren og returneres.
F1.2	Funksjonalitet	Sletting av videorom	Sletter ett spesifikt videorom
F1.3	Funksjonalitet	Få informasjon om ett videorom	Returnere informasjon om ett spesifikt videorom, f.eks antall deltagere, kvalitet ol
F1.4	Funksjonalitet	Starte en videostrøm	
F1.5	Funksjonalitet	Stoppe en videostrøm	Avslutte/stoppe strømming til ett rom/strøm
F1.6	Funksjonalitet		
I1.1	Infrastruktur	Contrainerisering	Mellomvaren skal kunne kjøres som Docker-image
I1.2	Infrastuktur	Skalerbarhet	Mellomvaren skal være stateless (ingen state kun i minne per instans). Dersom state er nødvendig lagres dette i en delt komponent (f.eks. Redis eller en database) Konfig skal komme via miljøvariabler (environment variables).
O1.1	Observability	Logging	Systemet skal logge sikkerhets- og driftsrelevante hendelser (mislykkede autentiseringsforsøk, opprett/slett rom, start/stop strøm), uten å logge hemmeligheter eller sensitiv informasjon.

O1.2	Observability	Helsesjekk	API-et skal eksponere endepunktene: /health/live (kjører API-et?) /health/ready (er API-et klar til å motta requests?)
O1.3	Observability	Logging/overvåking	Det er ønskelig at API-et har mulighet til å publisere statistiske data til prometheus. Eksempel på slik data kan være aktive videorom totalt, aktive videorom per backend osv.
D1.1	Dokumentasjon	Det må opprettes en dokumentasjon, som kan deles med leverandører for implementering.	

API dokumentasjon:

AntMedia:

<https://docs.antmedia.io/guides/developer-sdk-and-api/rest-api-guide/rest-apis-examples/>

<https://antmedia.io/rest/#/>

NHN:

<https://hjelp.video.nhn.no/space/IS/177012751/Booking+API+-+Dokumentasjon>

<https://developer.pexip.com/>

Rammeverk:

Det er ett ønske fra utviklermiljøet i HDO at det benyttes .NET 10.

Dette er ikke ett absolutt krav.

Tilgang til test miljøer:

AntMedia:

<https://bifrost-dev.hdo.no>

Sikkerhet: JWT token

NHN:

<https://join.test.video.nhn.no/>

Sikkerhet: Bruker/pwd

Begge miljøer er tilgjengelig over internett.

Det er krav til autentisering mot begge miljøer, dette utveksles på annet vis.

Appendix:

Appendix 1 – liste over brukte API endepunkt

Application API:

/application/rest/v2/broadcasts/create POST
/application/rest/v2/broadcasts/{stream_id} PUT
/application/rest/v2/broadcasts/{stream_id} DELETE
/application/rest/v2/broadcasts/list/{offset}/size GET
/application/rest/v2/broadcasts/count GET
/application/rest/v2/broadcasts/{stream_id}/rtmp-endpoint POST
/application/rest/v2/broadcasts/{stream_id}/rtmp-endpoint?endpointServiceId={stream_id} DELETE
/application/rest/v2/broadcasts/{stream_id}/start POST
/application/rest/v2/broadcasts/{stream_id}/stop POST
/application/rest/v2/vods/{vod_id} DELETE
/application/rest/v2/filters/create POST
/application/rest/v2/filters/list/{offset}/size GET
/application/rest/v2/filters/{filter_id} DELETE

Management API:

/rest/v2/applications/{application} POST
/rest/v2/applications GET
/rest/v2/applications-info/ GET
/rest/v2/applications/{application}?deleteDB=true DELETE
/rest/v2/applications/settings/{application} POST
/rest/v2/applications/settings/{application} GET

Referanser

Referanser

- [1] Helsetjenestens driftsorganisasjon for nødnett HF. Kravspesifikasjon: Hdo videoapi. Kravspesifikasjon, HDO, 2026. Se Vedlegg B.
- [2] NTNU og HDO. Samarbeidsavtale - bachelorprosjekt videoapi, 2026. Signert 18. januar 2026. Se Vedlegg A.